LABORATORY REPORT: LAB 5

Static Code Analysis & Remediation

Automated Code Quality, Security Vulnerability, and PEP 8 Style Auditing

Student Name:	Raghav Budur	**SRN / Roll:**	PES1UG23AM225 / Sl. No: 01
Course:	Software Engineering Lab (SE Lab)	**Target Script:**	library_system.py
Assigned Repo:	SETAPESU26/L5_01_library_fines	**User Repo:**	https://github.com/RAGHAV-BUDUR/SE-LAB
Tools Used:	Pylint 4.1.3, Bandit 1.9.4, Flake8 7.4.1	**Environment:**	Python 3.11.9 (Windows x64)

EXECUTIVE SUMMARY & KEY RESULTS

- **Pylint Score:** Improved from **6.07/10** to a **PERFECT 10.00/10** (+3.93 increase).
- **Bandit Security:** Decreased from **2 issues** (Medium eval vulnerability & Low try-except-pass) to **0 issues**.
- **Flake8 Style:** Resolved all **14 violations** (E722, E302, E305, E501, F401) to **0 violations (Exit code 0)**.
- **Verification:** Suite of 8 unit tests created in test_library_system.py, passing at 100%.

1. Objectives and Static Analysis Methodology

The primary objective of Lab 5 is to systematically detect, analyze, and rectify common programming bugs, security vulnerabilities, resource leaks, and style violations in the assigned library_system.py program using three industry-standard Python static analysis tools:

- **Pylint:** Inspects abstract syntax trees to flag code smells, mutable default parameters, unused variables, global variable mutations, and unclosed resources.
- **Bandit:** Scans Python source code to detect security vulnerabilities, dangerous built-ins (such as eval()), and insecure error-suppression patterns mapped to CWEs.
- **Flake8:** Enforces PEP 8 adherence, combining PyFlakes, pycodestyle, and McCabe metrics to guarantee consistent whitespace, line lengths, and syntax conventions.

2. Before vs After Static Analysis Metrics

Tool	Metric / Check	Baseline (Before)	Remediated (After)	Status
Pylint	Code Quality Score	**6.07 / 10**	**10.00 / 10**	**Resolved (+3.93)**
Bandit	Security Findings	2 issues (1 Med B307, 1 Low B110)	**0 issues**	**100% Clean**
Flake8	PEP 8 Violations	14 violations (E722, E302, E501...)	**0 violations**	**100% Compliant**
pytest	Functional Unit Tests	No automated tests	**8 tests passing**	**Verified**

3. Detailed Issue Table and Remediation Summary

Below is the comprehensive audit table documenting all genuine issues identified across Pylint, Bandit, and Flake8, their root cause, and the actual architectural and syntactic fixes implemented:

ID & Tool	Sev/Type	Line	Problem Description	Implemented Fix & Verification
W0102 Pylint	Medium Bug Risk	L9	**Mutable default argument:** dates=[] is evaluated once at definition time, causing timestamps to accumulate across separate student calls.	Replaced default with dates=None and allocated if dates is None: dates = [] inside method. **Verified:** Independent timestamps; W0102 resolved.
B307 / W0123 Bandit/Pylint	Medium CWE-78/95	L45	**Remote Code Execution:** eval() allowed arbitrary code injection via string interpolation in fine_from_rule().	Implemented safe recursive AST evaluator _eval_ast_node() restricting operations to basic math and days. **Verified:** Malicious input rejected; B307 eliminated.
B110 / E722 Bandit/Flake8	Low/Med Anti-pattern	L21	**Bare except pass:** try ... except: pass in return_book() silently swallowed interrupts and unexpected errors.	Replaced with explicit conditional dictionary checking (if student_id not in loans: return False). **Verified:** Deterministic boolean return; B110 & E722 resolved.
W0603 Pylint	Low Logic Flaw	L28, 35	**Global state mutation:** renewBook mutated global FINE_PER_DAY permanently; load_records used global rebinding.	Removed global mutation in renewal; used in-place dict mutation loans.clear(); loans.update(data) in file loader. **Verified:** Global rate preserved at 2/day.
R1732 / W1514 Pylint	Low Resource	L34, 40	**Unmanaged file handles:** open() without with context managers risks leaked file descriptors; missing encoding.	Refactored to with open(file, mode, encoding='utf-8') as f: context managers. **Verified:** Safe resource closure guaranteed; R1732 & W1514 resolved.
C0103 Pylint	Low Naming	L27	**Naming violation:** Function name renewBook violated PEP 8 snake_case naming style.	Renamed function to renew_book(). **Verified:** C0103 resolved.
C0114/C0116 Pylint	Low Docstring	Mod ule	**Missing docstrings:** Module and all functions lacked PEP 257 compliant documentation.	Added detailed module docstring and function docstrings explaining arguments and behaviors. **Verified:** Full documentation compliance.
E501 / E302 Flake8	Low Formatting	Multi ple	**PEP 8 violations:** Missing 2 blank lines between functions; lines exceeding 79 chars; unused import os.	Enforced standard 2-line spacing, wrapped long lines under 79 characters, removed unused import. **Verified:** Flake8 reported 0 errors.

4. Technical Reflection & Software Engineering Insights

Reflection 1: Tool Roles and Complementary Synergy
Static analysis provides a multi-layered defense mechanism. Pylint provides in-depth semantic code review, identifying design smells, mutable default parameters, and unclosed resources. Bandit operates as a dedicated application security scanner, successfully catching severe CWE-78/95 vulnerabilities like arbitrary code execution via eval(). Flake8 acts as an objective linter enforcing stylistic consistency and readability. Together, they eliminate entire classes of bugs long before runtime execution.

Reflection 2: Dangers of Mutable Defaults
Python evaluates default argument expressions once when the function is defined. Binding a mutable object like a list (dates=[]) causes that list to be shared across all invocations that omit the argument. This introduces subtle, stateful cross-call contamination. Using dates=None and initializing inside the function guarantees a clean, isolated list instance per invocation.

Reflection 3: Safe AST Evaluation vs. Arbitrary Evaluation
Using eval() to calculate fines from user rules opened a critical security hole where attackers could execute system-level commands. By implementing a recursive AST parser (_eval_ast_node), the application explicitly whitelists only mathematical binary operations and numeric constants. Any unsupported function call, import, or dangerous token immediately triggers a ValueError, achieving complete functional utility with zero security exposure.

Reflection 4: Shift-Left in CI/CD Pipelines
Integrating static analysis gates into automated CI/CD workflows ensures that code quality and security standards are enforced objectively. Catching defects during local pre-commit hooks or automated PR validation reduces the cost of defect remediation by orders of magnitude compared to debugging in staging or production environments.

5. Git Version Control History (Lab 5 Commits)

Commit Hash	Commit Message	Scope & Staged Files
d5a135a	fix(lab5): resolve pylint code quality issues	Resolved mutable defaults, docstrings, resource handling in library_system.py
c1ed769	fix(lab5): address bandit security findings	Replaced eval() with safe AST evaluator, removed bare except pass
e7eedab	fix(lab5): resolve flake8 formatting violations	Resolved PEP 8 line lengths and blank lines; committed reports, tests, and documentation

6. Deliverables & Verification Checklist

Deliverable Requirement	Artifact Location	Verification Status
Cleaned & Updated Source File	library_system.py	**Pylint 10.00 / Bandit 0 / Flake8 0**
Before Tool Reports	reports/lab5/before/	**Pylint (6.07), Bandit, Flake8 captured**
After Tool Reports	reports/lab5/after/	**Pylint (10.00), Bandit (0), Flake8 (0)**
Documented Issue Table	lab5_issue_table.md / issues.md	**8 comprehensive entries**
Technical Reflection Document	lab5_reflection.md / reflection.md	**Fully answered 4 core questions**
Automated Unit Test Suite	test_library_system.py	**8 tests passing (100%)**
Three Sequential Commits	Git branch main	**Commits d5a135a, c1ed769, e7eedab**